In [9]:
```
import pandas as pd
import numpy as np

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)

# ==========================
# 1. 读取清洗后的最终建模数据
# ==========================
df_model = pd.read_csv("cleaned_model_data.csv")

# 日期列转回 datetime
df_model["MthCalDt"] = pd.to_datetime(df_model["MthCalDt"])

print("Original df_model shape:", df_model.shape)
print("Original date range:", df_model["MthCalDt"].min(), "to", df_model[

# ==========================
# 2. 严格排除疫情影响：只保留 2022-01-01 及以后数据
# ==========================
df_model_post2022 = df_model[df_model["MthCalDt"] >= "2022-01-01"].copy()

print("\nPost-2022 df shape:", df_model_post2022.shape)
print("Post-2022 date range:", df_model_post2022["MthCalDt"].min(), "to",

# 特征列和目标列
feature_cols = ["Mkt-RF", "SMB", "HML", "RMW", "CMA"]
target_col = "y_next"

# 可用月份序列
all_months_post2022 = sorted(df_model_post2022["DateKey"].unique())

print("\nNumber of post-2022 months:", len(all_months_post2022))
print("First 5 months:", all_months_post2022[:5])
print("Last 5 months:", all_months_post2022[-5:])

display(df_model_post2022.head())
```

```
Original df_model shape: (1590627, 22)
Original date range: 1996-09-30 00:00:00 to 2025-12-31 00:00:00

Post-2022 df shape: (264908, 22)
Post-2022 date range: 2022-01-31 00:00:00 to 2025-12-31 00:00:00

Number of post-2022 months: 48
First 5 months: [np.int64(202201), np.int64(202202), np.int64(202203), np.
int64(202204), np.int64(202205)]
Last 5 months: [np.int64(202508), np.int64(202509), np.int64(202510), np.i
nt64(202511), np.int64(202512)]
```

	PERMNO	HdrCUSIP	Ticker	PERMCO	MthCalDt	MthPrc	MthRet	MthV
992	10026	46603210	JJSF	7976	2022-01-31	151.69	-0.039694	1487824.
993	10026	46603210	JJSF	7976	2022-02-28	163.72	0.079306	1727130.
994	10026	46603210	JJSF	7976	2022-03-31	155.10	-0.048821	1486734.
995	10026	46603210	JJSF	7976	2022-04-29	149.70	-0.034816	1212454.
996	10026	46603210	JJSF	7976	2022-05-31	128.22	-0.143487	2829929.

In [12]:
```
# ==========================
# 3. rolling 参数
# ==========================
window_size = 36            # 用过去36个月训练
candidate_pool_size = 100   # 每月流动性前100只股票
top_n = 6                   # 最终选6只股票

# ==========================
# 4. 结果容器
# ==========================
rf_monthly_results = []          # 每个月组合层面的结果
rf_stock_selection = []          # 每个月最终 top 6 股票
rf_candidate_predictions = []    # 每个月 candidate pool 的预测结果

print("Parameters initialized.")
```

```
Parameters initialized.
```

In [13]:
```
for i in range(window_size - 1, len(all_months_post2022) - 1):
    current_month = all_months_post2022[i]
    next_month = all_months_post2022[i + 1]
    train_months = all_months_post2022[i - window_size + 1 : i + 1]

    # ==========================
    # 1. 构造训练集：过去36个月
    # ==========================
    train_data = df_model_post2022[df_model_post2022["DateKey"].isin(trai

    if train_data.empty:
        continue

    X_train = train_data[feature_cols]
    y_train = train_data[target_col]

    # ==========================
    # 2. 当前月候选股票：按 DollarVol 取前100
    # ==========================
    current_data = df_model_post2022[df_model_post2022["DateKey"] == curr

    if current_data.empty:
        continue

    candidate_data = (
        current_data
```

```
        .sort_values("DollarVol", ascending=False)
        .head(candidate_pool_size)
        .copy()
    )

    if len(candidate_data) < top_n:
        continue

    # ==========================
    # 3. 训练 Random Forest
    # ==========================
    rf_model = RandomForestRegressor(
        n_estimators=200,
        max_depth=5,
        min_samples_leaf=10,
        random_state=42,
        n_jobs=-1
    )

    rf_model.fit(X_train, y_train)

    # ==========================
    # 4. 对 candidate pool 做预测
    # ==========================
    candidate_data["rf_pred"] = rf_model.predict(candidate_data[feature_c

    # 保存 candidate pool 预测结果
    candidate_output = candidate_data[
        ["PERMNO", "MthCalDt", "DateKey", "DollarVol", "y_next"]
    ].copy()
    candidate_output["rebalance_month"] = current_month
    candidate_output["holding_month"] = next_month
    candidate_output["rf_pred"] = candidate_data["rf_pred"]

    rf_candidate_predictions.append(candidate_output)

    # ==========================
    # 5. 选出预测值最高的 top 6 股票
    # ==========================
    selected = (
        candidate_data
        .sort_values("rf_pred", ascending=False)
        .head(top_n)
        .copy()
    )

    selected["rebalance_month"] = current_month
    selected["holding_month"] = next_month
    selected["rank_in_portfolio"] = range(1, len(selected) + 1)

    rf_stock_selection.append(
        selected[
            [
                "rebalance_month", "holding_month", "PERMNO", "MthCalDt",
                "rf_pred", "y_next", "DollarVol", "rank_in_portfolio"
            ]
        ].copy()
    )

    # ==========================
```

```
    # 6. 先计算 top 6 股票的等权组合收益
    # 后面如果要接 MVO，再替换成优化权重收益
    # ==========================
    ew_return = selected["y_next"].mean()

    rf_monthly_results.append({
        "rebalance_month": current_month,
        "holding_month": next_month,
        "n_train_obs": len(train_data),
        "n_candidates": len(candidate_data),
        "n_selected": len(selected),
        "rf_ew_return": ew_return
    })

print("Rolling Random Forest on post-2022 data completed.")
```

```
Rolling Random Forest on post-2022 data completed.
```

In [14]:

```
# ==========================
# 5. 拼接结果表
# ==========================
rf_monthly_results_df = pd.DataFrame(rf_monthly_results)

rf_stock_selection_df = (
    pd.concat(rf_stock_selection, ignore_index=True)
    if rf_stock_selection else pd.DataFrame()
)

rf_candidate_predictions_df = (
    pd.concat(rf_candidate_predictions, ignore_index=True)
    if rf_candidate_predictions else pd.DataFrame()
)

print("rf_monthly_results_df shape:", rf_monthly_results_df.shape)
print("rf_stock_selection_df shape:", rf_stock_selection_df.shape)
print("rf_candidate_predictions_df shape:", rf_candidate_predictions_df.s

display(rf_monthly_results_df.head())
display(rf_stock_selection_df.head(12))
display(rf_candidate_predictions_df.head(12))
```

```
rf_monthly_results_df shape: (12, 6)
rf_stock_selection_df shape: (72, 8)
rf_candidate_predictions_df shape: (1200, 8)
```

	rebalance_month	holding_month	n_train_obs	n_candidates	n_selected	rf_ew_r
0	202412	202501	198728	100	6	0.0
1	202501	202502	199268	100	6	-0.0
2	202502	202503	199788	100	6	-0.06
3	202503	202504	200283	100	6	0.0
4	202504	202505	200822	100	6	0.0

	rebalance_month	holding_month	PERMNO	MthCalDt	rf_pred	y_next	
0	202412	202501	93436	2024-12-31	0.033954	-0.001818	7.6
1	202412	202501	11308	2024-12-31	0.033954	0.015895	2.4
2	202412	202501	18163	2024-12-31	0.033954	-0.007530	2.2
3	202412	202501	14702	2024-12-31	0.033954	0.105259	2.2
4	202412	202501	38703	2024-12-31	0.033954	0.118168	2.2
5	202412	202501	20357	2024-12-31	0.033954	-0.058285	2.3
6	202501	202502	86580	2025-01-31	0.018532	0.037093	7.0
7	202501	202502	18726	2025-01-31	0.018532	-0.024427	2.5
8	202501	202502	11762	2025-01-31	0.018532	-0.104758	2.3
9	202501	202502	14541	2025-01-31	0.018532	0.071612	2.4
10	202501	202502	14702	2025-01-31	0.018532	-0.124846	2.4
11	202501	202502	86783	2025-01-31	0.018532	0.055475	2.4

	PERMNO	MthCalDt	DateKey	DollarVol	y_next	rebalance_month	holdin
0	93436	2024-12-31	202412	7.651331e+11	-0.001818	202412	
1	84398	2024-12-31	202412	6.210318e+11	0.023156	202412	
2	86580	2024-12-31	202412	5.461313e+11	-0.109590	202412	
3	86755	2024-12-31	202412	3.190814e+11	0.017934	202412	
4	14593	2024-12-31	202412	2.431685e+11	-0.061283	202412	
5	93002	2024-12-31	202412	2.182147e+11	-0.049292	202412	
6	10107	2024-12-31	202412	1.840053e+11	-0.018979	202412	
7	84788	2024-12-31	202412	1.659304e+11	0.079668	202412	
8	13407	2024-12-31	202412	1.533532e+11	0.173359	202412	
9	86211	2024-12-31	202412	1.508484e+11	0.152263	202412	
10	19788	2024-12-31	202412	1.494702e+11	0.087005	202412	
11	88222	2024-12-31	202412	1.277369e+11	0.021282	202412	

In [16]:

```
# ==========================
# 把每个月 top 6 股票整理成宽表
# 一行 = 一个月
# ==========================

# 先保证按月份和排名排序
top6_wide = rf_stock_selection_df[
    ["rebalance_month", "holding_month", "rank_in_portfolio", "PERMNO", "
].sort_values(["rebalance_month", "rank_in_portfolio"]).copy()

# pivot 成宽表
top6_permno = top6_wide.pivot(
    index=["rebalance_month", "holding_month"],
    columns="rank_in_portfolio",
    values="PERMNO"
)

top6_pred = top6_wide.pivot(
    index=["rebalance_month", "holding_month"],
    columns="rank_in_portfolio",
    values="rf_pred"
)

top6_actual = top6_wide.pivot(
    index=["rebalance_month", "holding_month"],
```

```
    columns="rank_in_portfolio",
    values="y_next"
)

# 重命名列名
top6_permno.columns = [f"stock_{i}" for i in top6_permno.columns]
top6_pred.columns = [f"pred_{i}" for i in top6_pred.columns]
top6_actual.columns = [f"actual_{i}" for i in top6_actual.columns]

# 合并成一个表
top6_summary_by_month = pd.concat(
    [top6_permno, top6_pred, top6_actual],
    axis=1
).reset_index()

# 保存
top6_summary_by_month.to_csv("rf_top6_by_month_post2022.csv", index=False

print("Saved: rf_top6_by_month_post2022.csv")
display(top6_summary_by_month.head())
```

```
Saved: rf_top6_by_month_post2022.csv
```

	rebalance_month	holding_month	stock_1	stock_2	stock_3	stock_4	stock_5
0	202412	202501	93436	11308	18163	14702	38703
1	202501	202502	86580	18726	11762	14541	14702
2	202502	202503	86580	78975	57665	86456	75186
3	202503	202504	84398	66181	65875	66093	89071
4	202504	202505	84398	92027	77178	57665	89469

In [17]:

```
# ==========================
# 6. 预测误差评估
# 用 candidate pool 中股票的 y_next 和 rf_pred 比较
# ==========================
pred_eval = rf_candidate_predictions_df.dropna(subset=["y_next", "rf_pred

rf_rmse = np.sqrt(mean_squared_error(pred_eval["y_next"], pred_eval["rf_p
rf_mae = mean_absolute_error(pred_eval["y_next"], pred_eval["rf_pred"])

print("Random Forest Prediction Performance (Post-2022 only)")
print("RMSE:", round(rf_rmse, 6))
print("MAE :", round(rf_mae, 6))
```

```
Random Forest Prediction Performance (Post-2022 only)
RMSE: 0.112015
MAE : 0.073849
```

In [19]:

```
# ==========================
# 7. 组合表现评估（先看等权）
# ==========================
portfolio_returns = rf_monthly_results_df["rf_ew_return"].dropna()

annualized_return = (1 + portfolio_returns).prod() ** (12 / len(portfolio
annualized_vol = portfolio_returns.std() * np.sqrt(12)

if portfolio_returns.std() != 0:
    sharpe_ratio = (portfolio_returns.mean() / portfolio_returns.std()) *
```

```
else:
    sharpe_ratio = np.nan

wealth = (1 + portfolio_returns).cumprod()
peak = wealth.cummax()
drawdown = (peak - wealth) / peak
max_drawdown = drawdown.max()

print("Random Forest Equal-Weight Portfolio Performance (Post-2022 only)"
print("Annualized Return    :", round(annualized_return, 6))
print("Annualized Volatility:", round(annualized_vol, 6))
print("Sharpe Ratio         :", round(sharpe_ratio, 6))
print("Max Drawdown         :", round(max_drawdown, 6))
```

```
Random Forest Equal-Weight Portfolio Performance (Post-2022 only)
Annualized Return    : 0.210254
Annualized Volatility: 0.129452
Sharpe Ratio         : 1.54534
Max Drawdown         : 0.08036
```

In [20]:

```
 # ==========================
# 8. 保存结果
# ==========================
rf_monthly_results_df.to_csv("rf_monthly_results_post2022.csv", index=Fal
rf_stock_selection_df.to_csv("rf_top6_stock_selection_post2022.csv", inde
rf_candidate_predictions_df.to_csv("rf_candidate_predictions_post2022.csv

print("Saved files:")
print("- rf_monthly_results_post2022.csv")
print("- rf_top6_stock_selection_post2022.csv")
print("- rf_candidate_predictions_post2022.csv")
```

```
Saved files:
- rf_monthly_results_post2022.csv
- rf_top6_stock_selection_post2022.csv
- rf_candidate_predictions_post2022.csv
```